Deep Dive 02 | Authentication and User Data

Understand the User schema, password lifecycle, JWT contract and Google OAuth boundary.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. User document

backend/models/User.js defines name, username, email, password, profilePic, role and optional googleId, with timestamps. Username and email are unique. Role is constrained to student or teacher. The model uses bcryptjs to hash the password before saving and exposes matchPassword and getSignedJwtToken methods.

2. Password registration and login

Step	Registration	Login
Input	Multipart form fields and optional image.	JSON email and password.
Lookup	Checks existing email and username.	Finds user by email.
Secret handling	Mongoose pre-save hashes password.	bcrypt comparePassword checks submitted value.
Response	Returns selected user fields and signed token.	Returns selected user fields and signed token.

3. JWT protected requests

The model signs an id claim using JWT_SECRET and a one-hour expiry. The frontend attaches the token as Authorization: Bearer. authMiddleware verifies signature and expiry, loads the associated user excluding password, and passes control to the route. student and teacher guards check the role attached to req.user.

4. Google OAuth

When GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET are present, Passport configures a Google strategy. A new Google user is saved with a generated username, student role and placeholder password. The callback creates an application JWT and redirects to the frontend auth-success route. This means OAuth and application authorization are two sequential layers.

5. Important security review

- AuthContext persists user data and JWT in localStorage. XSS could expose the token.
- Google callback carries token and user ID in query parameters; URLs can appear in history, logs or referrers.
- Refresh code decodes an expired JWT payload then uses the identifier to look up a user and issue a new token; decoded data itself is not signature verification.
- Multer file validation checks extensions and supplied MIME type; size/signature limits should be strengthened.
- Some auth handlers log submitted or identity data. Ensure secrets and passwords are always redacted.

6. Safer next design

Use a short-lived access token in an HttpOnly, Secure, SameSite cookie or a one-time code exchange after OAuth. Keep refresh tokens signed, revocable, rotated and stored hashed. Add CSRF protection if cookies are used, request validation, login rate limits, explicit OAuth state checks, file size/signature validation and structured redacted logs.

7. What to say in an interview

“Passwords are hashed with bcryptjs and protected API routes verify signed JWTs. Google OAuth establishes identity through Passport, after which the app issues its own token. The current browser token-storage and redirect design are improvement areas, so I would move toward secure cookies or one-time code exchange and a revocable refresh flow.”